

Department of Computer Science and Engineering
Assignment Questions

Course Code & Name: CSA0303 – Data Structures – Slot C

Assignment Title:	Design and implementation of a Web Browser with Stack Data Structures.
Course Outcome(s) – CO:	CO1: Comprehend the fundamentals of linear and non-linear data structures and their abstract data types. CO2: Compare the performance of various searching techniques (Linear Search, Binary Search) and understand the application of different linear data structures such as Linked Lists, Stacks, and Queues.
Bloom's Taxonomy Level	L3- Apply L4 – Analyze
SDG Mapping (SDG 1-17)	SDG 9 - Industry, Innovation, and Infrastructure SDG 11 – Sustainable Cities and Communities;

1. Problem Statement:

A web browser needs to efficiently manage webpage navigation and text-editing operations using the Stack data structure. The browser must support Back and Forward navigation, Undo and Redo operations, and the situation where a user visits a completely new webpage after moving backward. Since stacks follow the Last-In-First-Out (LIFO) principle, they are suitable for storing the most recently visited webpages and editing actions. The design must determine how stack operations such as Push, Pop, and Clear are used for each browser operation and analyze whether a single stack is sufficient to manage all these functions.

2. Objective:

- To understand the application of **stacks in web browser operations**.
- To design Back and Forward navigation using stacks.
- To design Undo and Redo operations using stacks.
- To understand the effect of opening a new webpage after navigating backward.
- To analyze why multiple stacks are required instead of a single stack.
- To demonstrate the practical application of the **LIFO principle**.

3. Requirements and Environment Used:

- Hardware: Any standard computer/laptop.
- Operating System: Windows.
- Programming Language: C.
- Compiler: Standard C compiler.
- Editor/IDE: Visual Studio Code
- Data Structures: Stack

4. Design / Proposed Solution:

4.1 Back Navigation

When the user clicks **Back**:

1. The current webpage is pushed into the Forward Stack.
2. The top webpage is popped from the Back Stack.
3. The popped webpage becomes the current webpage.

4.2 Forward Navigation

When the user clicks **Forward**:

1. The current webpage is pushed into the Back Stack.
2. The top webpage is popped from the Forward Stack.
3. It becomes the current webpage.

4.3 Visiting a New Webpage

When a new webpage is opened:

1. The current webpage is pushed into the Back Stack.
2. The new webpage becomes the current webpage.

3. The Forward Stack is **cleared**.

For example:

A → B → C → Back → New Page D

Before visiting D:

- Back Stack: A, B
- Current: C → Back → B
- Forward Stack: C

After visiting D:

- Back Stack: A, B
- Current: D
- Forward Stack: Empty

Therefore, webpage C can no longer be accessed using Forward.

4.3 Undo and Redo

- Every new editing action is pushed onto the **Undo Stack**.
- Undo pops an action from Undo and pushes it onto Redo.
- Redo pops an action from Redo and pushes it back onto Undo.
- A new edit after Undo clears the Redo Stack.

4.4 Is One Stack Sufficient?

No. A single stack cannot independently manage Back, Forward, Undo, and Redo operations. Each operation requires its own history. Therefore, the proposed design uses **four logical stacks**.

5. Algorithm / Pseudocode / Flowchart

5.1 Visiting a New Webpage

1. Start.
2. Enter the new webpage.
3. Push the current webpage into the Back Stack.
4. Update the current webpage.
5. Clear the Forward Stack.
6. Display the current webpage.
7. Stop.

5.2 Back Operation

1. Start.
2. Check whether Back Stack is empty.
3. If empty, display "**No previous page.**"
4. Otherwise, push the current page into Forward Stack.
5. Pop the top element from Back Stack.
6. Make it the current page.
7. Stop.

5.3 Forward Operation

1. Start
2. Check whether Forward Stack is empty.
3. If empty, display "**No forward page.**"
4. Otherwise, push the current page into Back Stack.
5. Pop the top element from Forward Stack.
6. Make it the current page.
7. Stop.

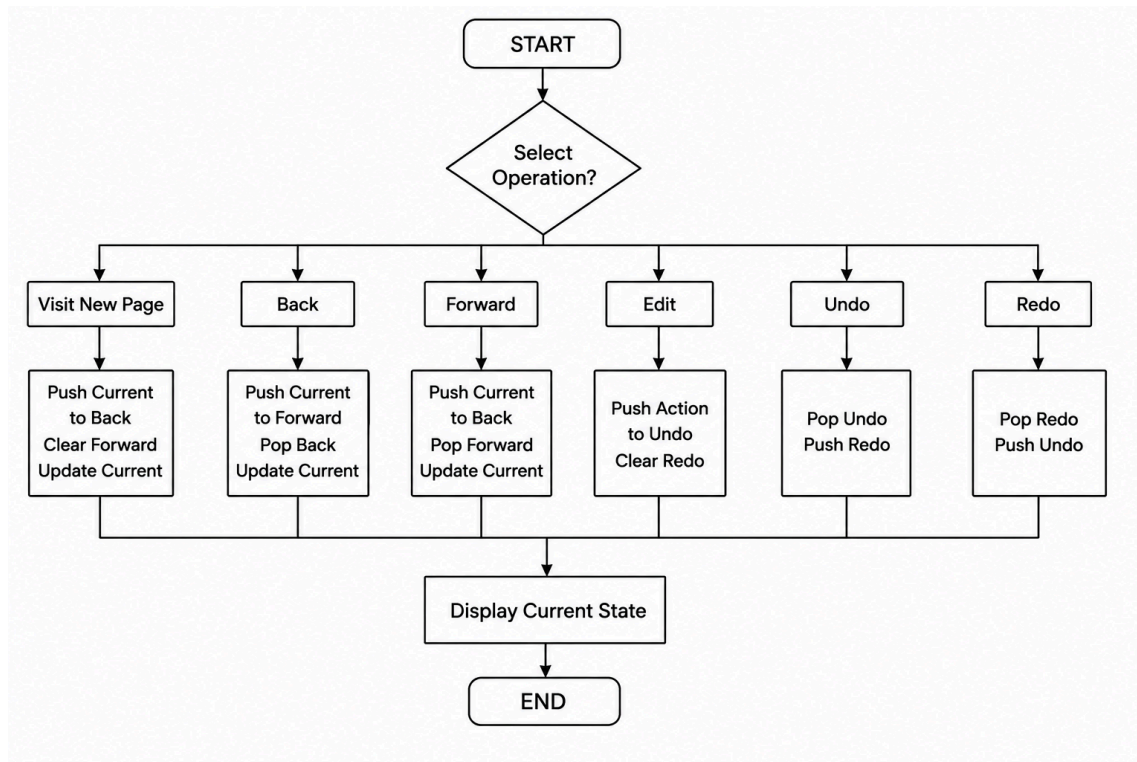
5.4 Undo

1. Check whether Undo Stack is empty.
2. If empty, display "Nothing to undo."
3. Otherwise, pop the top action from Undo Stack.
4. Push it into Redo Stack.
5. Stop

5.4 Redo

1. Check whether Redo Stack is empty.
2. If empty, display "**Nothing to redo.**"
3. Otherwise, pop the top action from Redo Stack.
4. Push it into Undo Stack.
5. Stop.

5.5 Flowchart



6. Implementation / Source Code

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define MAX 50
```

```
#define SIZE 100
```

```
// Stack structure
```

```
typedef struct {  
    char data[MAX][SIZE];  
    int top;  
} Stack;
```

```
// Initialize stack
```

```
void init(Stack *s) {  
    s->top = -1;  
}
```

// Check if stack is empty

```
int isEmpty(Stack *s) {  
    return s->top == -1;  
}
```

// Push an item

```
void push(Stack *s, char item[]) {  
    if (s->top == MAX - 1) {  
        printf("Stack is full!\n");  
        return;  
    }
```

```
    s->top++;  
    strcpy(s->data[s->top], item);  
}
```

// Pop an item

```
void pop(Stack *s, char item[]) {  
    if (isEmpty(s)) {  
        return;  
    }
```

```
    strcpy(item, s->data[s->top]);  
    s->top--;  
}
```

```
// Clear stack
```

```
void clear(Stack *s) {  
    s->top = -1;  
}
```

```
// Display stack
```

```
void displayStack(Stack *s) {  
    if (isEmpty(s)) {  
        printf("Empty");  
        return;  
    }
```

```
    for (int i = s->top; i >= 0; i--) {  
        printf("%s", s->data[i]);
```

```
        if (i != 0)  
            printf(" <- ");
```

```
    }  
}
```

```
int main() {
```

```
    Stack backStack;
```

```
    Stack forwardStack;
```

```
    Stack undoStack;
```

```
    Stack redoStack;
```

```
    char currentPage[SIZE] = "Home";
```

```

char page[SIZE];
char action[SIZE];

int choice;

// Initialize all stacks
init(&backStack);
init(&forwardStack);
init(&undoStack);
init(&redoStack);

do {

    printf("\n=====\\n");
    printf("    WEB BROWSER SYSTEM\\n");
    printf("=====\\n");

    printf("Current Page : %s\\n", currentPage);

    printf("\\n1. Visit New Page");
    printf("\\n2. Back");
    printf("\\n3. Forward");
    printf("\\n4. Edit Text");
    printf("\\n5. Undo");
    printf("\\n6. Redo");
    printf("\\n7. Display Stacks");
    printf("\\n8. Exit");

    printf("\\n\\nEnter your choice: ");

```

```

scanf("%d", &choice);

// -----
// 1. Visit New Page
// -----
if (choice == 1) {

    printf("Enter webpage name: ");
    scanf("%99s", page);

    // Current page goes to Back Stack
    push(&backStack, currentPage);

    // New page becomes current
    strcpy(currentPage, page);

    // Forward history is cleared
    clear(&forwardStack);

    printf("\nVisited: %s\n", currentPage);
    printf("Forward history cleared.\n");
}

// -----
// 2. Back
// -----
else if (choice == 2) {

    if (isEmpty(&backStack)) {

```

```

        printf("\nNo previous page available.\n");

    } else {

        // Current page goes to Forward Stack
        push(&forwardStack, currentPage);

        // Previous page becomes current
        pop(&backStack, currentPage);

        printf("\nBack to: %s\n", currentPage);
    }
}

// -----
// 3. Forward
// -----
else if (choice == 3) {

    if (isEmpty(&forwardStack)) {

        printf("\nNo forward page available.\n");

    } else {

        // Current page goes to Back Stack
        push(&backStack, currentPage);

```

```
    // Forward page becomes current
    pop(&forwardStack, currentPage);

    printf("\nForward to: %s\n", currentPage);
}
}
```

```
// -----
// 4. Edit Text
// -----
```

```
else if (choice == 4) {
```

```
    printf("Enter editing action: ");
    scanf("%99s", action);
```

```
    // Save action in Undo Stack
    push(&undoStack, action);
```

```
    // New edit removes Redo history
    clear(&redoStack);
```

```
    printf("\nEdit saved: %s\n", action);
}
```

```
// -----
// 5. Undo
// -----
```

```
else if (choice == 5) {
```

```

if (isEmpty(&undoStack)) {

    printf("\nNothing to undo.\n");

} else {

    // Move action from Undo to Redo
    pop(&undoStack, action);
    push(&redoStack, action);

    printf("\nUndone: %s\n", action);
}
}

// -----
// 6. Redo
// -----
else if (choice == 6) {

    if (isEmpty(&redoStack)) {

        printf("\nNothing to redo.\n");

    } else {

        // Move action from Redo to Undo
        pop(&redoStack, action);
        push(&undoStack, action);
    }
}

```

```

        printf("\nRedone: %s\n", action);
    }
}

// -----
// 7. Display Stacks
// -----
else if (choice == 7) {

    printf("\n===== CURRENT STATE =====\n");

    printf("Current Page : %s\n", currentPage);

    printf("Back Stack  : ");
    displayStack(&backStack);

    printf("\nForward Stack : ");
    displayStack(&forwardStack);

    printf("\nUndo Stack   : ");
    displayStack(&undoStack);

    printf("\nRedo Stack    : ");
    displayStack(&redoStack);

    printf("\n===== \n");
}

// -----

```

```

// Invalid choice
// -----
else if (choice != 8) {

    printf("\nInvalid choice!\n");
}

} while (choice != 8);

printf("\nBrowser closed successfully.\n");

return 0;
}

```

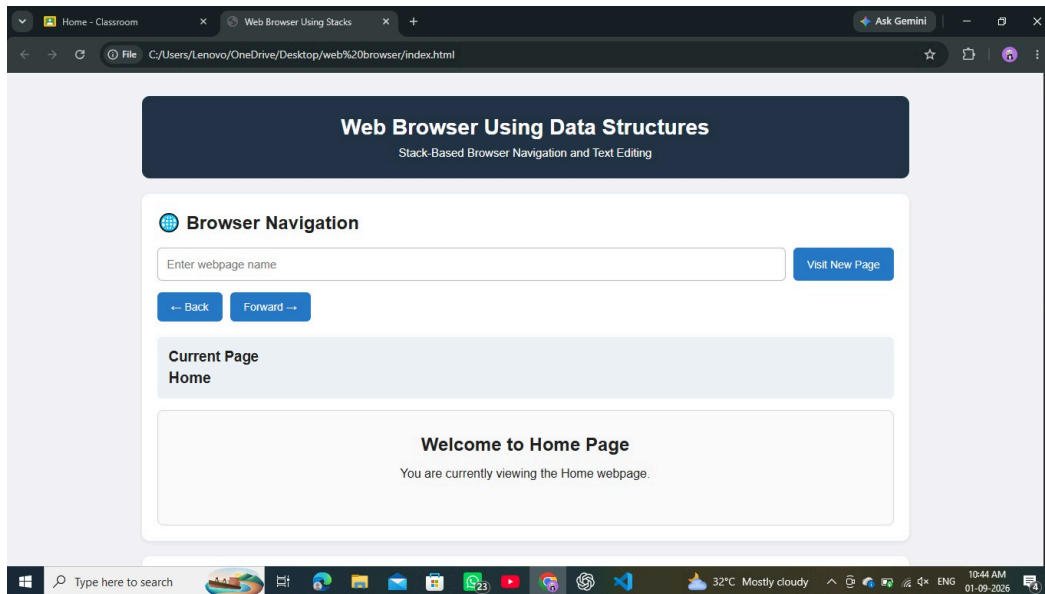
7. Test Cases and Expected/Actual Results

Test Case	Operation	Expected Result	Actual Result
TC1	Visit A → B → C	Current page = C	Successful
TC2	Back from C	Current page = B	Successful
TC3	Forward from B	Current page = C	Successful
TC4	A → B → C → Back → D	Forward Stack becomes empty	Successful
TC5	Edit X → Edit Y → Undo	Y moves to Redo Stack	Successful
TC6	Undo → Redo	Y is restored	Successful
TC7	Back with empty stack	No previous page message	Successful
TC8	Forward with empty stack	No forward page message	Successful

8. Execution Screenshots / Output

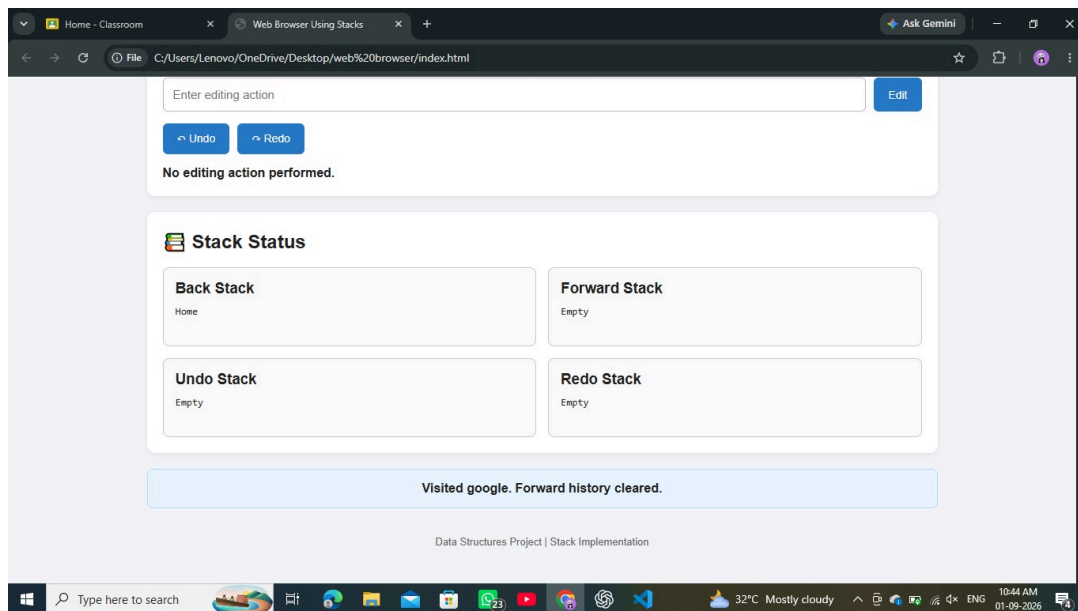
Screenshot 1

Initial browser state showing:



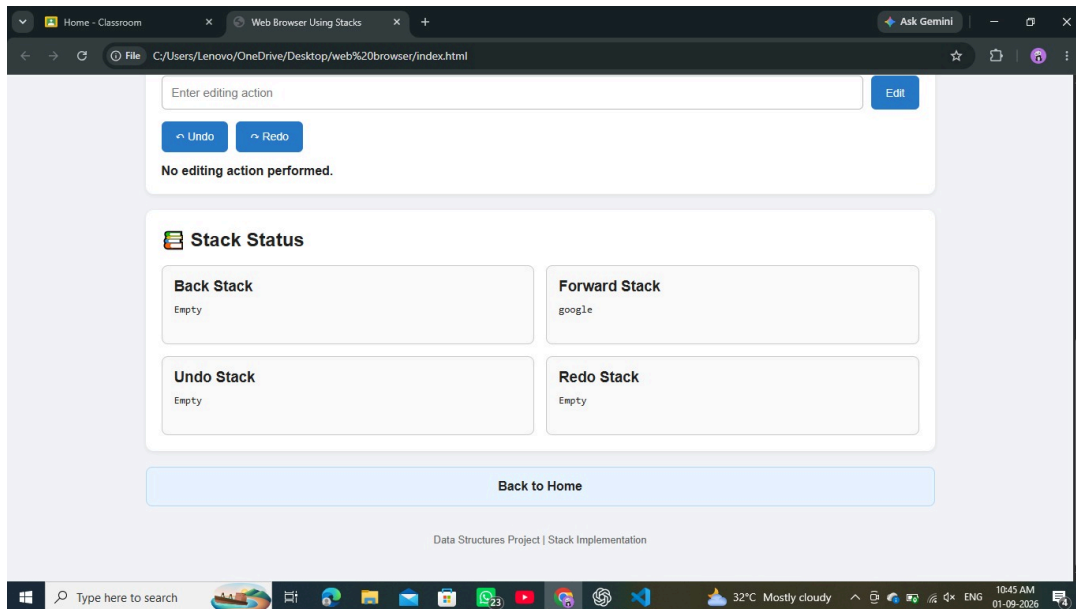
Screenshot 2

After visiting webpages:



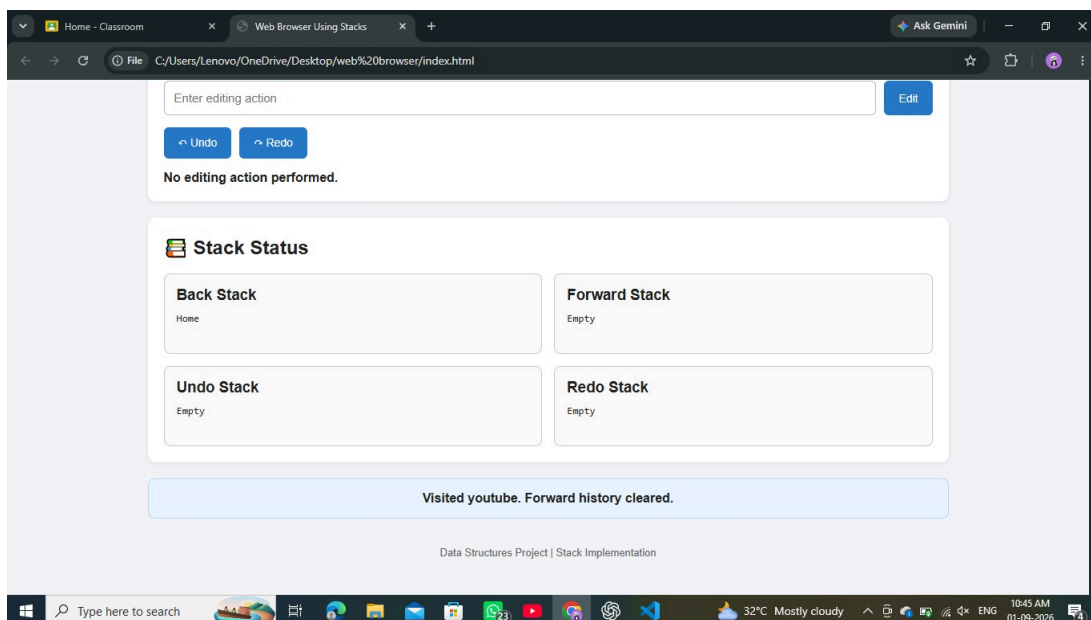
Screenshot 3

After clicking Back:



Screenshot 4

After visiting a new webpage following Back:



9. Analysis and Discussion

The Stack data structure is highly suitable for browser navigation because it follows the LIFO principle. The most recently visited webpage is the first webpage that should be accessed when the Back operation is performed.

A single stack is not sufficient because Back and Forward operations need to maintain two different histories. Similarly, Undo and Redo require separate stacks to keep track of reversible and restored actions.

The use of stacks makes insertion and deletion efficient because both Push and Pop operations take $O(1)$ time. The space complexity is $O(n)$, where n represents the number of webpages or editing actions stored.

When a user moves backward and then visits a new webpage, the Forward Stack is cleared. This prevents the browser from continuing along an old navigation path.

Thus, the design requires:

4 Logical Stacks $\rightarrow$ Back + Forward + Undo + Redo

10. Conclusion

The web browser can be effectively modeled using the Stack data structure. Back and Forward navigation can be managed using two stacks, while Undo and Redo operations can be handled using another two stacks. The LIFO property of stacks makes these operations simple and efficient. When a user navigates backward and opens a new webpage, the Forward Stack is cleared and a new browsing path is created. Therefore, one stack is not sufficient, and multiple stacks are required for complete browser functionality.

11. Individual Contribution of Group Members

S. No.	Name of the Member	Register Number	Contribution
1	Harinika . S	192521444	Implementation of the stack-based web browser operations
2	M. Rumana khan	192471025	Documentation, report preparation, and presentation

12. References

1. Ellis Horowitz, Sartaj Sahni, and Susan Anderson-Freed, Fundamentals of Data Structures.
2. Thomas H. Cormen et al., Introduction to Algorithms.
3. E.Balagurusamy, “Data Structures Using C”, 2nd Edition, McGraw Hill Education, 2017
4. 3. M. A. Weiss, "Data Structures and Algorithm Analysis in C",, 2nd ed.,Pearson Education Asia, 2007.

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Requirements Analysis (CO1)	15	Clearly identifies all requirements: large graph, weighted roads, traffic updates, 200 ms response, memory constraints, and repeated queries	Identifies most requirements with minor omissions	Identifies basic routing requirements only	Limited understanding of the problem
Min-Heap, HashSet & HashMap Integration (CO4)	25	Efficiently integrates a Min-Heap with Dijkstra; correctly performs insertion, extraction, and priority updates	Min-Heap integrated correctly with minor issues	Priority queue used but inefficiently implemented	No effective priority queue/heap implementation

Dijkstra's Shortest Path Algorithm (CO5)	25	Correctly implements Dijkstra's algorithm for weighted roads and returns the optimal available route	Algorithm works with minor implementation issues	Basic implementation works for simple cases	Basic implementation works for simple cases
--	----	--	--	---	---

C Program, Performance, Testing & Complexity Analysis (CO1/CO2)	25	Complete, modular, error-free C program with appropriate structures, functions, memory management, and meaningful comments	Well-structure d program with minor errors	Functional but poorly structured or partially incomplete	Major errors/incomplete program
Reflection: Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of design choices, clear connection to SDG 9 and 11 relevance, and honest, specific account of challenges faced and learning gained.	General justification of design choices; sustainability connection and challenges mentioned but lack depth.	Reflection present but generic; limited justification of design or vague account of learning outcomes.	No genuine reflection submitted, or content is generic with no real design/SDG/learn ing connection.

Deliverables

To receive full credit, each submission must include the following deliverables:

1. Pseudo code
2. Implementation & Results
3. Github Upload